# ALERTA: Diseño, Auditoría e Ingeniería Inversa de un Mini-Lenguaje de Dominio Específico para la Alerta Temprana de Deslizamientos

**Proyecto Final**

**Valery Andrea Ortiz Mesa**

**1044213851**

**Jhonatan David Vivas Arango**

**1124312515**

**Programa de Ingeniería de Sistemas y Computación**

**Teoría de Lenguajes Formales**

**Juan Manuel Cardona Arias**

## Introducción

En los Andes colombianos, las lluvias intensas y de corta duración suelen detonar movimientos en masa (Aristizábal et al., 2022), y la relación entre lluvia y deslizamientos se estudia desde Caine (1980). La Ley 1523 de 2012 define la alerta como el estado que se declara antes de un evento peligroso, con base en el monitoreo del fenómeno, para que las entidades y la población activen procedimientos establecidos (Congreso de la República de Colombia, 2012, art. 4). Los avisos públicos del IDEAM expresan esos estados por colores (La FM, 2017).

Esa definición describe un programa: leer un sensor, evaluar una condición y activar un procedimiento. ALERTA es un lenguaje de dominio específico (DSL) para escribir esas reglas de modo que una máquina pueda validarlas antes de usarlas, porque un error de escritura podría silenciar una evacuación. El objetivo es diseñarlo desde sus bases matemáticas y construir su analizador sintáctico en Python, demostrando el control sobre la ambigüedad y sobre las transiciones de estados.

## Descripción del Lenguaje ALERTA

Un programa declara sus sensores y, después de `inicio`, escribe sus reglas con variables, expresiones, condicionales y ciclos. Los niveles de alerta son `AMARILLA`, `NARANJA` y `ROJA`. Este es un ejemplo:

```
# Regla de alerta para una ladera. Los umbrales son ilustrativos.
programa LADERA_NORTE

sensor PLUVIOMETRO;
sensor HUMEDAD;

inicio
    limite = 40 + 12 * 3;
    lluvia = leer PLUVIOMETRO;
    si lluvia > limite y leer HUMEDAD >= 85 entonces
        alertar ROJA;
    sino
        esperar 10;
    fin
fin
```

El alcance es léxico y sintáctico. ALERTA no predice deslizamientos ni calibra umbrales, y los valores de los ejemplos son ilustrativos. Tampoco comprueba reglas semánticas, por ejemplo que un sensor leído esté declarado.

## Fase 1: Formalismo Léxico

Se distinguen dos niveles. En el nivel léxico, los símbolos son caracteres del alfabeto Σ. En el nivel sintáctico, los símbolos son tokens, que forman el conjunto de terminales T de la gramática. El AFD de esta fase convierte caracteres en tokens.

### Definición del Alfabeto (Σ)

Sean L = {a, …, z, ñ}, U = {A, …, Z, Ñ}, D = {0, …, 9}, E = {espacio, tabulador, retorno de carro}, N = {salto de línea} y A = {á, é, í, ó, ú, ü, Á, É, Í, Ó, Ú, Ü, ¿, ¡}. Si P es el conjunto de los 95 caracteres ASCII imprimibles, entonces:

$$\Sigma = P \cup L \cup U \cup E \cup N \cup A, \quad |\Sigma| = 114$$

El conteo es 95 de P, más ñ y Ñ, más 14 de A, más tabulador, retorno de carro y salto de línea. Para simplificar la matriz, Σ se divide en las 16 clases de la Tabla 1, que agrupan símbolos con igual comportamiento. Un carácter fuera de Σ es un error léxico.

**Tabla 1**

*Clases de símbolos de Σ*

| Clase | Símbolos | Card. | Clase | Símbolos | Card. |
|---|---|---|---|---|---|
| `l` | L: minúsculas | 27 | `*/` | * y / | 2 |
| `U` | U: mayúsculas | 27 | `(` | paréntesis izquierdo | 1 |
| `d` | D: dígitos | 10 | `)` | paréntesis derecho | 1 |
| `_` | guion bajo | 1 | `;` | punto y coma | 1 |
| `=` | igual | 1 | `#` | numeral | 1 |
| `<>` | < y > | 2 | `esp` | E: espacios | 3 |
| `!` | admiración | 1 | `nl` | N: salto de línea | 1 |
| `+-` | + y - | 2 | `otro` | resto de Σ | 33 |

## Expresiones Regulares de los Tokens

La Tabla 2 define cada token con unión (|), concatenación y estrella de Kleene (∗).

**Tabla 2**

*Expresiones regulares de los tokens*

| Token | Expresión regular | Ejemplo |
|---|---|---|
| ID | L (L \| D \| _)∗ − R | `lluvia`, `suelo_2` |
| NOMBRE | U (U \| D \| _)∗ − R | `PLUVIOMETRO` |
| NUM | D D∗ | `85` |
| OPREL | == \| != \| <= \| >= \| < \| > | `>=` |
| OPSUMA | + \| - | `+` |
| OPMUL | * \| / | `*` |
| =, (, ), ; | Cada símbolo es su propio token | `;` |
| Palabras reservadas (R) | Cada palabra es su propio token | `sensor`, `ROJA` |
| COMENTARIO | # (Σ − N)∗ | `# nota` |
| ESPACIO | (E \| N) (E \| N)∗ | espacios |

*Nota.* R = {programa, sensor, inicio, fin, alertar, esperar, si, entonces, sino, mientras, hacer, leer, o, y, AMARILLA, NARANJA, ROJA}, con |R| = 17. COMENTARIO y ESPACIO se reconocen y se descartan: no llegan a la gramática.

ID y NOMBRE excluyen R, y esa diferencia sigue siendo regular porque R es finito (Hopcroft et al., 2006). Para no crear un estado por cada prefijo de las palabras reservadas, se usa la técnica habitual de una tabla (Aho et al., 2006): el AFD acepta la palabra en q1 o q2 y luego se consulta R.

## Diseño Formal del AFD

El autómata es la tupla M = (Q, Σ, δ, q0, F), con:

$$Q = \{q0, q1, \ldots, q15, qE\}, \quad |Q| = 17$$

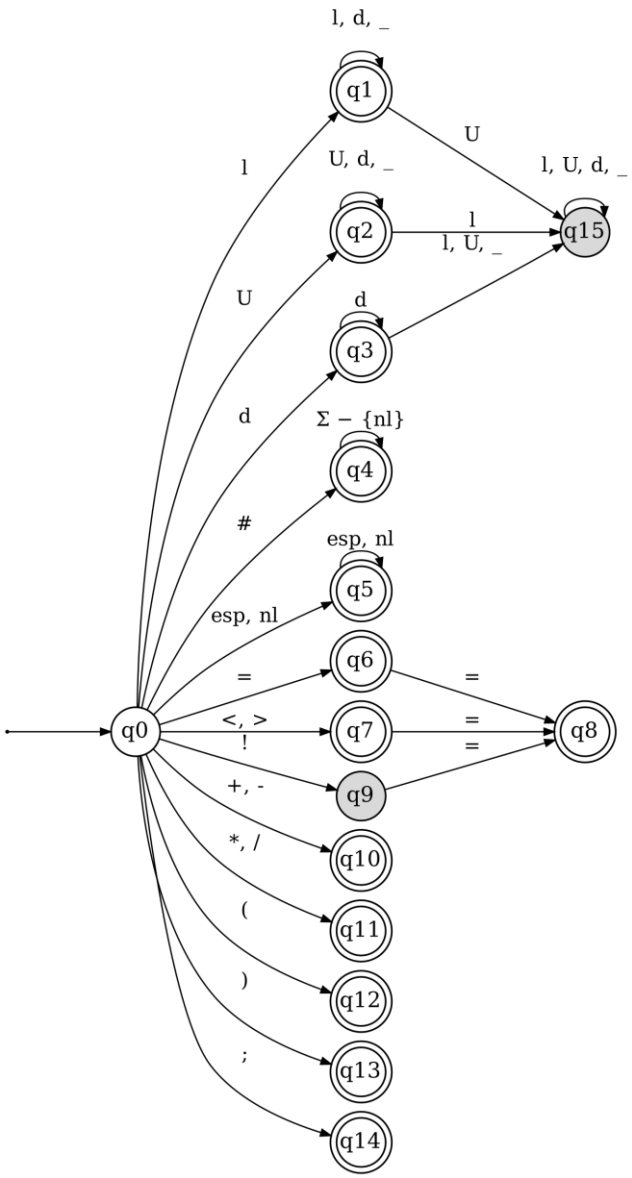
$$F = \{q1, \ldots, q8, q10, \ldots, q14\}, \quad |F| = 13$$

$$Q - F = \{q0, q9, q15, qE\}$$

La función $\delta : Q \times \Sigma \rightarrow Q$ es total y determinista, y se define por clases en la Tabla 3. El analizador avanza hasta que $\delta$ lleva a qE, sin consumir ese símbolo. Si el último estado está en F, emite el token; si no, reporta un error. La Figura 1 muestra el grafo.

**Figura 1**

*Grafo del AFD de ALERTA*




*Nota.* El doble círculo indica aceptación y el círculo gris, rechazo. La flecha sin origen marca q0. Se omiten qE y las transiciones hacia él, que sí están en la Tabla 3.

**Tabla 3**

*Matriz de transiciones δ del AFD*

| δ | l | U | d | _ | = | <> | ! | +- | */ | ( | ) | ; | # | esp | nl | otro |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| → q0 | q1 | q2 | q3 | E | q6 | q7 | q9 | q10 | q11 | q12 | q13 | q14 | q4 | q5 | q5 | E |
| ∗ q1 | q1 | q15 | q1 | q1 | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q2 | q15 | q2 | q2 | q2 | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q3 | q15 | q15 | q3 | q15 | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | q4 | E | q4 |
| ∗ q5 | E | E | E | E | E | E | E | E | E | E | E | E | E | q5 | q5 | E |
| ∗ q6 | E | E | E | E | q8 | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q7 | E | E | E | E | q8 | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q8 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| q9 | E | E | E | E | q8 | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q10 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q11 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q12 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q13 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| ∗ q14 | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |
| q15 | q15 | q15 | q15 | q15 | E | E | E | E | E | E | E | E | E | E | E | E |
| qE | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E | E |

*Nota.* → estado inicial; ∗ estado de aceptación; E = qE.

## Justificación de los Estados de Aceptación y Rechazo

Un estado es de aceptación si lo leído es un lexema completo. Es de rechazo si lo leído no forma un token. La Tabla 4 justifica cada estado.

**Tabla 4**

*Justificación de los estados*

| Estado | Tipo | Salida | Justificación |
|---|---|---|---|
| q0 | Rechazo | L01 | Inicial: lo leído es ε, que no es un token |
| q1 | Aceptación | ID o reservada | Palabra en minúsculas; se consulta R |
| q2 | Aceptación | NOMBRE o reservada | Palabra en mayúsculas; se consulta R |
| q3 | Aceptación | NUM | Uno o más dígitos |
| q4 | Aceptación | COMENTARIO | Se descarta; termina en el salto de línea |
| q5 | Aceptación | ESPACIO | Se descarta |
| q6 | Aceptación | = | Asignación; otro = lleva a q8 |
| q7 | Aceptación | OPREL | < o > completos; un = posterior lleva a q8 |
| q8 | Aceptación | OPREL | ==, !=, <= o >=: no admiten continuación |
| q9 | Rechazo | L02 | ! solo no es token; necesita = |
| q10 a q14 | Aceptación | OPSUMA, OPMUL, (, ), ; | Un estado por token, en ese orden |
| q15 | Rechazo | L03 | Palabra mal formada, como `Hola` o `12abc` |
| qE | Rechazo | L04 o fin de token | Trampa: ninguna continuación produce un token |

Por ejemplo, `>=` se acepta: δ(q0, >) = q7 y δ(q7, =) = q8, con salida OPREL. En cambio, `12a` se rechaza: δ(q0, 1) = q3, δ(q3, 2) = q3 y δ(q3, a) = q15, que no pertenece a F. Estas trazas se obtienen con la opción `--traza`.

Se comprobó que los 17 estados son alcanzables y que ninguno sobra si cada estado se distingue por su token o diagnóstico (Hopcroft et al., 2006). Si solo importara aceptar o rechazar, q15 y qE serían equivalentes y bastarían 16 estados; q15 se conserva porque permite consumir toda la palabra defectuosa y dar un mensaje preciso. Un script auxiliar, que no forma parte de la entrega, comparó el AFD con las expresiones regulares en 6.728.903 cadenas de hasta 5 símbolos, sin discrepancias.

## Fase 2: Estructura Sintáctica

La sintaxis se define con una gramática independiente del contexto (GIC), que es una tupla $G = (N, \Sigma_T, S, P)$: N es el conjunto de no terminales, $\Sigma_T$ el de terminales, $S \in N$ el símbolo inicial

y P el conjunto de producciones. Se escribe en formato Backus-Naur (BNF), donde `<x>` es un no terminal, el texto entre comillas es un token literal y ε es la cadena vacía.

Los terminales de la gramática son los tokens de la Fase 1, no los caracteres. Por eso se escribe $\Sigma_T$, para no confundirlo con el alfabeto de caracteres Σ: para el AFD, `alertar` son siete símbolos de Σ; para la gramática, es un solo terminal de $\Sigma_T$.

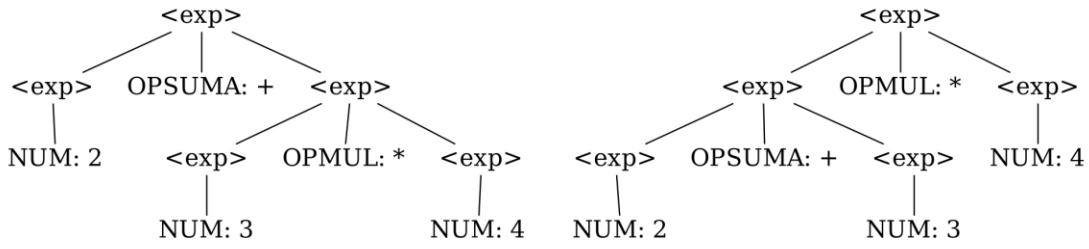
**Gramática Inicial G0**

```
<programa>    ::= "programa" NOMBRE <sensores> "inicio" <sentencias> "fin"
<sensores>    ::= <sensores> "sensor" NOMBRE ";" | ε
<sentencias>  ::= <sentencias> <sentencia> | ε
<sentencia>   ::= ID "=" <exp> ";"
                | "alertar" <nivel> ";"
                | "esperar" <exp> ";"
                | "si" <exp> "entonces" <sentencias> "fin"
                | "si" <exp> "entonces" <sentencias> "sino" <sentencias> "fin"
                | "mientras" <exp> "hacer" <sentencias> "fin"
<nivel>       ::= "AMARILLA" | "NARANJA" | "ROJA"
<exp>         ::= <exp> "o" <exp> | <exp> "y" <exp> | <exp> OPREL <exp>
                | <exp> OPSUMA <exp> | <exp> OPMUL <exp>
                | "(" <exp> ")" | NUM | ID | "leer" NOMBRE
```

G0 es natural de escribir, pero tiene 27 conflictos en la tabla de análisis LL(1): es ambigua, tiene recursividad por la izquierda y tiene prefijos comunes. Una gramática es ambigua si alguna cadena tiene dos árboles de derivación (Sipser, 2013). La cadena `2 + 3 * 4` tiene los dos de la Figura 2, que dan 14 y 20.

**Figura 2**

*Dos árboles de derivación de 2 + 3 * 4 en G0*




*(a) Multiplicación primero: 2 + (3 * 4) = 14* *(b) Suma primero: (2 + 3) * 4 = 20*

**Paso 1: Eliminación de la Ambigüedad**

Se crea un no terminal por cada nivel de precedencia (Tabla 5). Un operador de menor precedencia solo puede quedar debajo de uno de mayor precedencia si se usan paréntesis. La recursión por la izquierda fija la asociatividad.

**Tabla 5**

*Jerarquía de operadores*

| Nivel | Operadores | No terminal | Asociatividad |
|---|---|---|---|
| 1 (menor) | `o` | `<exp>` | Izquierda |
| 2 | `y` | `<conj>` | Izquierda |
| 3 | `== != < <= > >=` | `<rel>` | No asociativo |
| 4 | `+ -` | `<suma>` | Izquierda |
| 5 | `* /` | `<prod>` | Izquierda |
| 6 (mayor) | números, variables, `leer`, `( )` | `<factor>` | No aplica |

La regla `<exp>` se reemplaza por las siguientes, y así se obtiene G1:

```
<exp>    ::= <exp> "o" <conj> | <conj>
<conj>   ::= <conj> "y" <rel> | <rel>
<rel>    ::= <suma> OPREL <suma> | <suma>
<suma>   ::= <suma> OPSUMA <prod> | <prod>
<prod>   ::= <prod> OPMUL <factor> | <factor>
<factor> ::= "(" <exp> ")" | NUM | ID | "leer" NOMBRE
```

Es la solución estándar E → E + T | T, T → T * F | F, F → (E) | id: `<suma>`, `<prod>` y `<factor>` cumplen el papel de E, T y F, y sobre ellos se agregan los niveles de las comparaciones, `y` y `o`. Como cada nivel solo contiene operandos del siguiente, `*` siempre queda más abajo que `+` en el árbol.

Este paso restringe el lenguaje a propósito: `a < b < c` deja de ser válido, porque `<rel>` no es asociativo. Los pasos siguientes sí conservan el lenguaje.

**Paso 2: Eliminación de la Recursividad por la Izquierda**

Un parser descendente no puede usar reglas como $A \rightarrow A\ \alpha$, porque expandiría A sin consumir entrada. Se aplica la transformación estándar (Aho et al., 2006):

$A \rightarrow A\ \alpha \mid \beta$ se convierte en $A \rightarrow \beta\ A'$ y $A' \rightarrow \alpha\ A' \mid \varepsilon$

**Lema 1.** La transformación conserva el lenguaje.

**Demostración.** En la forma original, A aplica $k \geq 0$ veces $A \rightarrow A\ \alpha$ y termina con $\beta$, de modo que genera $\beta\ \alpha^k$, es decir, $\beta\ \alpha*$. En la forma nueva, $A'$ genera $\alpha*$ y A genera $\beta$ seguido de $A'$, es decir, $\beta\ \alpha*$. Ambas generan el mismo lenguaje.

La Tabla 6 aplica la transformación a las seis reglas afectadas.

**Tabla 6**

*Eliminación de la recursividad por la izquierda*

| Regla en G1 | α | β | Resultado |
|---|---|---|---|
| `<sensores> ::= <sensores> "sensor" NOMBRE ";" \| ε` | `"sensor" NOMBRE ";"` | ε | `<sensores> ::= "sensor" NOMBRE ";" <sensores> \| ε` |
| `<sentencias> ::= <sentencias> <sentencia> \| ε` | `<sentencia>` | ε | `<sentencias> ::= <sentencia> <sentencias> \| ε` |
| `<exp> ::= <exp> "o" <conj> \| <conj>` | `"o" <conj>` | `<conj>` | `<exp> ::= <conj> <exp'>` `<exp'> ::= "o" <conj> <exp'> \| ε` |
| `<conj> ::= <conj> "y" <rel> \| <rel>` | `"y" <rel>` | `<rel>` | `<conj> ::= <rel> <conj'>` `<conj'> ::= "y" <rel> <conj'> \| ε` |
| `<suma> ::= <suma> OPSUMA <prod> \| <prod>` | `OPSUMA <prod>` | `<prod>` | `<suma> ::= <prod> <suma'>` `<suma'> ::= OPSUMA <prod> <suma'> \| ε` |
| `<prod> ::= <prod> OPMUL <factor> \| <factor>` | `OPMUL <factor>` | `<factor>` | `<prod> ::= <factor> <prod'>` `<prod'> ::= OPMUL <factor> <prod'> \| ε` |

*Nota.* Cuando $\beta = \varepsilon$, la regla $A \rightarrow A'$ es unitaria, y $A'$ se renombra como A.

**Paso 3: Factorización por la Izquierda**

Si dos alternativas comparten un prefijo, un token no basta para elegir. Las reglas $A \rightarrow \gamma\ \delta_1 \mid \gamma\ \delta_2$ se sustituyen por $A \rightarrow \gamma\ A'$ y $A' \rightarrow \delta_1 \mid \delta_2$ (Aho et al., 2006).

**Lema 2.** La factorización conserva el lenguaje.

**Demostración.** Ambas formas generan $\gamma(\delta_1 \mid \delta_2)$, por la distributividad de la concatenación sobre la unión.

La Tabla 7 muestra los dos casos.

**Tabla 7**

*Factorización por la izquierda*

| Prefijo común γ | Alternativas | Resultado |
|---|---|---|
| `"si" <exp> "entonces" <sentencias>` | `"fin"` y `"sino" <sentencias> "fin"` | `<sentencia> ::= "si" <exp> "entonces" <sentencias> <alternativa> "fin"`<br>`<alternativa> ::= "sino" <sentencias> \| ε` |
| `<suma>` | `OPREL <suma>` y ε | `<rel> ::= <suma> <rel'>`<br>`<rel'> ::= OPREL <suma> \| ε` |

## Gramática Final G

La gramática final tiene 17 no terminales, 27 terminales y 34 producciones:

```
<programa>    ::= "programa" NOMBRE <sensores> "inicio" <sentencias> "fin"
<sensores>    ::= "sensor" NOMBRE ";" <sensores> | ε
<sentencias>  ::= <sentencia> <sentencias> | ε
<sentencia>   ::= ID "=" <exp> ";"
                | "alertar" <nivel> ";"
                | "esperar" <exp> ";"
                | "si" <exp> "entonces" <sentencias> <alternativa> "fin"
                | "mientras" <exp> "hacer" <sentencias> "fin"
<alternativa> ::= "sino" <sentencias> | ε
<nivel>       ::= "AMARILLA" | "NARANJA" | "ROJA"
<exp>         ::= <conj> <exp'>
<exp'>        ::= "o" <conj> <exp'> | ε
<conj>        ::= <rel> <conj'>
<conj'>       ::= "y" <rel> <conj'> | ε
<rel>         ::= <suma> <rel'>
<rel'>        ::= OPREL <suma> | ε
<suma>        ::= <prod> <suma'>
<suma'>       ::= OPSUMA <prod> <suma'> | ε
<prod>        ::= <factor> <prod'>
<prod'>       ::= OPMUL <factor> <prod'> | ε
<factor>      ::= NUM | ID | "leer" NOMBRE | "(" <exp> ")"
```

Sus elementos, según la definición de GIC, son:

N = {`<programa>`, `<sensores>`, `<sentencias>`, `<sentencia>`, `<alternativa>`, `<nivel>`, `<exp>`, `<exp'>`, `<conj>`, `<conj'>`, `<rel>`, `<rel'>`, `<suma>`, `<suma'>`, `<prod>`, `<prod'>`, `<factor>`}, con |N| = 17.

$\Sigma_T$ = {(, ), ;, =, AMARILLA, ID, NARANJA, NOMBRE, NUM, OPMUL, OPREL, OPSUMA, ROJA, alertar, entonces, esperar, fin, hacer, inicio, leer, mientras, o, programa, sensor, si, sino, y}, con $|\Sigma_T| = 27$. El símbolo inicial es S = `<programa>`, y P contiene las 34 producciones escritas arriba, contando cada alternativa como una producción.

**Demostración de que G no es Ambigua**

Se demuestra que G es LL(1), porque ninguna gramática LL(1) es ambigua (Aho et al., 2006). La Tabla 8 presenta los conjuntos PRIMEROS y SIGUIENTES.

**Tabla 8**

*Conjuntos PRIMEROS y SIGUIENTES*

| No terminal | PRIMEROS | SIGUIENTES |
|---|---|---|
| <programa> | programa | $ |
| <sensores> | sensor, ε | inicio |
| <sentencias> | ID, alertar, esperar, mientras, si, ε | fin, sino |
| <sentencia> | ID, alertar, esperar, mientras, si | ID, alertar, esperar, fin, mientras, si, sino |
| <alternativa> | sino, ε | fin |
| <nivel> | AMARILLA, NARANJA, ROJA | ; |
| <exp> | (, ID, NUM, leer | ), ;, entonces, hacer |
| <exp'> | o, ε | ), ;, entonces, hacer |
| <conj> | (, ID, NUM, leer | ), ;, entonces, hacer, o |
| <conj'> | y, ε | ), ;, entonces, hacer, o |
| <rel> | (, ID, NUM, leer | ), ;, entonces, hacer, o, y |
| <rel'> | OPREL, ε | ), ;, entonces, hacer, o, y |
| <suma> | (, ID, NUM, leer | ), ;, OPREL, entonces, hacer, o, y |
| <suma'> | OPSUMA, ε | ), ;, OPREL, entonces, hacer, o, y |
| <prod> | (, ID, NUM, leer | ), ;, OPREL, OPSUMA, entonces, hacer, o, y |
| <prod'> | OPMUL, ε | ), ;, OPREL, OPSUMA, entonces, hacer, o, y |
| <factor> | (, ID, NUM, leer | ), ;, OPMUL, OPREL, OPSUMA, entonces, hacer, o, y |

*Nota.* $ marca el fin de la entrada.

Para cada alternativa, el conjunto de predicción es PRIMEROS(α) sin ε, más SIGUIENTES(A) si α puede derivar ε. La Tabla 9 muestra que las alternativas de cada no terminal tienen conjuntos disjuntos.

**Tabla 9**

*Conjuntos de predicción*

| No terminal | Alternativa | Predicción |
|---|---|---|
| <sensores> | sensor NOMBRE ; <sensores> | sensor |
| <sensores> | ε | inicio |
| <sentencias> | <sentencia> <sentencias> | ID, alertar, esperar, mientras, si |
| <sentencias> | ε | fin, sino |
| <sentencia> | ID = <exp> ; | ID |
| <sentencia> | alertar <nivel> ; | alertar |
| <sentencia> | esperar <exp> ; | esperar |
| <sentencia> | si <exp> entonces <sentencias> <alternativa> fin | si |
| <sentencia> | mientras <exp> hacer <sentencias> fin | mientras |
| <alternativa> | sino <sentencias> | sino |
| <alternativa> | ε | fin |
| <nivel> | AMARILLA | AMARILLA |
| <nivel> | NARANJA | NARANJA |
| <nivel> | ROJA | ROJA |
| <exp'> | o <conj> <exp'> | o |
| <exp'> | ε | ), ;, entonces, hacer |
| <conj'> | y <rel> <conj'> | y |
| <conj'> | ε | ), ;, entonces, hacer, o |
| <rel'> | OPREL <suma> | OPREL |
| <rel'> | ε | ), ;, entonces, hacer, o, y |
| <suma'> | OPSUMA <prod> <suma'> | OPSUMA |
| <suma'> | ε | ), ;, OPREL, entonces, hacer, o, y |
| <prod'> | OPMUL <factor> <prod'> | OPMUL |
| <prod'> | ε | ), ;, OPREL, OPSUMA, entonces, hacer, o, y |
| <factor> | NUM | NUM |
| <factor> | ID | ID |
| <factor> | leer NOMBRE | leer |
| <factor> | ( <exp> ) | ( |

**Teorema.** G no es ambigua.

**Demostración.** Sea w ∈ L(G). En cada paso de una derivación por la izquierda, la forma sentencial es x A γ, donde x es un prefijo de w. Sea a el símbolo de w que sigue a x. Como los conjuntos de predicción de A son disjuntos, solo una producción admite a. Por inducción, w tiene una sola derivación por la izquierda y un solo árbol.

## Derivaciones por la Izquierda y por la Derecha

Una derivación por la izquierda reemplaza en cada paso el no terminal que está más a la izquierda, y una por la derecha, el que está más a la derecha. Las Tablas 10 y 11 derivan `2 + 3 * 4` desde `<exp>` con la gramática G, ambas con las mismas 17 producciones, aplicadas en distinto orden. Los terminales se muestran con su lexema.

**Tabla 10**

*Derivación por la izquierda de 2 + 3 * 4 en G*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | <exp> | |
| 1 | <conj> <exp'> | <exp> → <conj> <exp'> |
| 2 | <rel> <conj'> <exp'> | <conj> → <rel> <conj'> |
| 3 | <suma> <rel'> <conj'> <exp'> | <rel> → <suma> <rel'> |
| 4 | <prod> <suma'> <rel'> <conj'> <exp'> | <suma> → <prod> <suma'> |
| 5 | <factor> <prod'> <suma'> <rel'> <conj'> <exp'> | <prod> → <factor> <prod'> |
| 6 | 2 <prod'> <suma'> <rel'> <conj'> <exp'> | <factor> → NUM |
| 7 | 2 <suma'> <rel'> <conj'> <exp'> | <prod'> → ε |
| 8 | 2 + <prod> <suma'> <rel'> <conj'> <exp'> | <suma'> → OPSUMA <prod> <suma'> |
| 9 | 2 + <factor> <prod'> <suma'> <rel'> <conj'> <exp'> | <prod> → <factor> <prod'> |
| 10 | 2 + 3 <prod'> <suma'> <rel'> <conj'> <exp'> | <factor> → NUM |
| 11 | 2 + 3 * <factor> <prod'> <suma'> <rel'> <conj'> <exp'> | <prod'> → OPMUL <factor> <prod'> |
| 12 | 2 + 3 * 4 <prod'> <suma'> <rel'> <conj'> <exp'> | <factor> → NUM |
| 13 | 2 + 3 * 4 <suma'> <rel'> <conj'> <exp'> | <prod'> → ε |
| 14 | 2 + 3 * 4 <rel'> <conj'> <exp'> | <suma'> → ε |
| 15 | 2 + 3 * 4 <conj'> <exp'> | <rel'> → ε |
| 16 | 2 + 3 * 4 <exp'> | <conj'> → ε |
| 17 | 2 + 3 * 4 | <exp'> → ε |

**Tabla 11**

*Derivación por la derecha de 2 + 3 * 4 en G*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | <exp> | |
| 1 | <conj> <exp'> | <exp> → <conj> <exp'> |
| 2 | <conj> | <exp'> → ε |
| 3 | <rel> <conj'> | <conj> → <rel> <conj'> |
| 4 | <rel> | <conj'> → ε |
| 5 | <suma> <rel'> | <rel> → <suma> <rel'> |
| 6 | <suma> | <rel'> → ε |
| 7 | <prod> <suma'> | <suma> → <prod> <suma'> |
| 8 | <prod> + <prod> <suma'> | <suma'> → OPSUMA <prod> <suma'> |
| 9 | <prod> + <prod> | <suma'> → ε |
| 10 | <prod> + <factor> <prod'> | <prod> → <factor> <prod'> |
| 11 | <prod> + <factor> * <factor> <prod'> | <prod'> → OPMUL <factor> <prod'> |
| 12 | <prod> + <factor> * <factor> | <prod'> → ε |
| 13 | <prod> + <factor> * 4 | <factor> → NUM |
| 14 | <prod> + 3 * 4 | <factor> → NUM |
| 15 | <factor> <prod'> + 3 * 4 | <prod> → <factor> <prod'> |
| 16 | <factor> + 3 * 4 | <prod'> → ε |
| 17 | 2 + 3 * 4 | <factor> → NUM |

Las dos derivaciones son distintas en el orden de los pasos, pero corresponden al mismo árbol: cada no terminal se expande con la misma producción en ambas. Esto no contradice el teorema, porque la ambigüedad se mide por árboles, no por el orden de las derivaciones. En G0, en cambio, la misma cadena tenía dos derivaciones por la izquierda distintas, con dos árboles distintos. En G, `*` queda debajo de `+`, y el valor es 14.

Un reconocedor auxiliar comparó G1 y G en 1.948.717 expresiones de hasta 6 tokens y en 3.693 programas aleatorios, sin diferencias, lo que respalda los Lemas 1 y 2.

## Fase 3: Implementación del Parser

El programa `alerta_parser.py` recibe la ruta de un archivo de texto y genera el árbol de derivación sintáctica o la lista de errores. Usa solo Python estándar, sin `re` ni generadores de parsers. Tiene tres partes:

La **Parte 1** es el analizador léxico. La matriz de la Tabla 3 está escrita en el código como `MATRIZ`, y la función `analizar_lexico` la recorre. La **Parte 2** es un parser descendente recursivo: cada no terminal de G es una función, y cada función lleva su regla BNF como comentario. Para elegir la alternativa, cada función mira el token actual, lo que es válido porque G es LL(1). Por ejemplo:

```
# <suma'> ::= OPSUMA <prod> <suma'> | ε
def suma_prima(self):
    if self.tipo() != "OPSUMA":
        return epsilon("<suma'>")
    return Nodo("<suma'>", [self.consumir("OPSUMA"), self.prod(), self.suma_prima()])
```

La **Parte 3** imprime el árbol o los errores, y contiene la batería de pruebas. Estas son las órdenes:

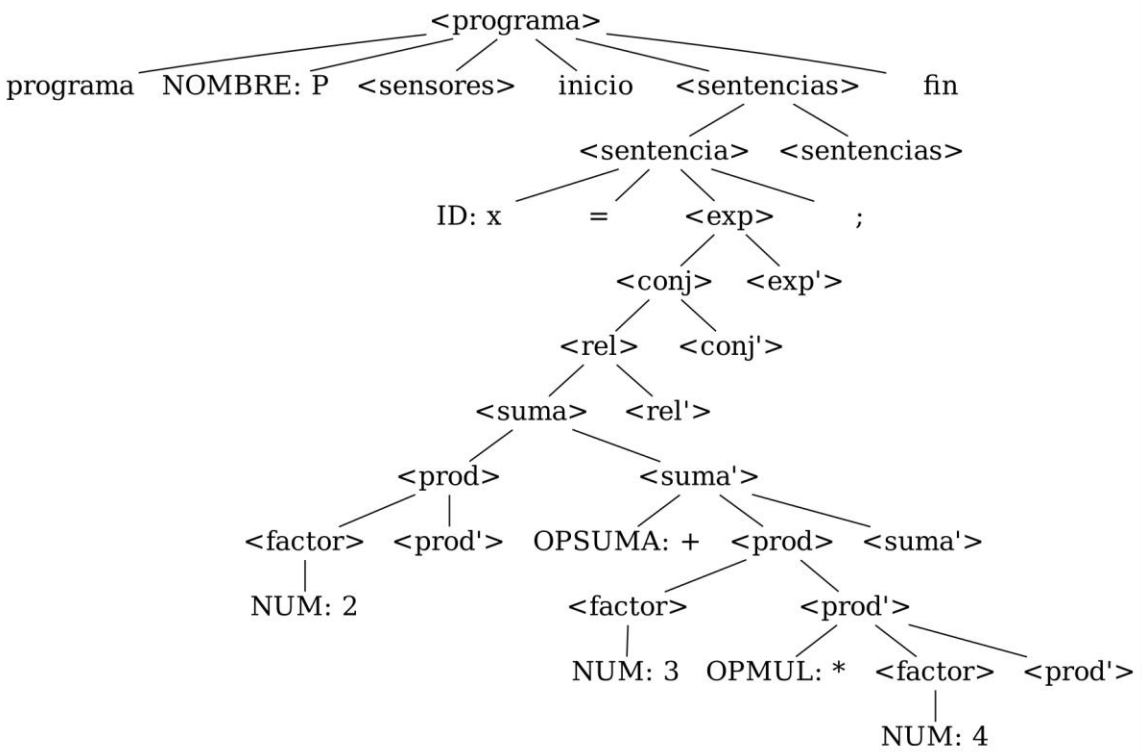
```
python alerta_parser.py programa.alerta            # árbol o errores
python alerta_parser.py programa.alerta --tokens   # además, lista de tokens
python alerta_parser.py programa.alerta --traza    # además, pasos del AFD
python alerta_parser.py --pruebas                  # batería de pruebas
```

**Árbol de Derivación Sintáctica**

La Figura 3 muestra el árbol de `programa P inicio x = 2 + 3 * 4; fin`, sin las hojas ε. La raíz es el símbolo inicial `<programa>`, los nodos internos son no terminales y cada nodo con sus hijos corresponde a una producción. Las hojas, leídas de izquierda a derecha, forman de nuevo la cadena de tokens del programa. El programa imprime este mismo árbol en la consola.

**Figura 3**

*Árbol de derivación de un programa de ejemplo*




## Manejo de Errores

El programa no se detiene en el primer error. La Tabla 12 presenta todos los códigos.

**Tabla 12**

*Códigos de error*

| Código | Origen | Causa | Ejemplo |
|---|---|---|---|
| L01 | q0 → qE | Símbolo que no inicia un token | `x = 2 @;` |
| L02 | q9 | `!` sin = | `x = a ! b;` |
| L03 | q15 | Palabra mal formada | `x = 12abc;` |
| L04 | Fuera de Σ | Símbolo que no pertenece al alfabeto | `x = 2 €;` |
| S01 | `consumir` | Llegó un token distinto al esperado | `x = 1 esperar 2;` |
| S02 | Sin alternativa | Ninguna alternativa admite el token | `alertar VERDE;` |

Tras un error sintáctico se aplica el modo pánico (Aho et al., 2006): se saltan tokens hasta el fin de la sentencia o el inicio de otra, y el análisis continúa. El archivo `ejemplo_errores.alerta` produce esta salida:

```
LISTA DE ERRORES (7)
  [S01] línea 4, columna 8: se esperaba NOMBRE y se encontró ';'
  [S02] línea 8, columna 13: se esperaba un valor y se encontró '*'
  [S01] línea 10, columna 5: se esperaba ; y se encontró 'alertar'
  [S02] línea 10, columna 13: nivel de alerta no válido: 'VERDE'
  [L02] línea 11, columna 11: '!' aislado: el operador válido es '!=': '!'
  [L03] línea 12, columna 9: palabra mal formada: mezcla mayúsculas, minúsculas o dígit...
  [S01] línea 16, columna 1: se esperaba fin y se encontró 'fin de archivo'

RESULTADO: programa RECHAZADO.
```

**Pruebas**

La opción `--pruebas` ejecuta 16 casos: 6 válidos y 10 inválidos, y todos dan el resultado esperado. El archivo `ejemplo_valido.alerta`, de 80 tokens, se acepta. Además, un script auxiliar comparó el parser con un análisis LL(1) estricto en 813.616 secuencias cortas de tokens y en 37.642 programas alterados: coincidieron siempre al aceptar o rechazar. Otras 5.000 versiones dañadas del ejemplo no provocaron fallos.

**Modificación del Lenguaje**

Para permitir variables que empiecen por _, basta cambiar δ(q0, _) de E a q1 en `MATRIZ`. Para agregar la sentencia `silenciar;`, se agrega la palabra a `RESERVADAS` y un caso en la función `sentencia`. Ambos cambios se probaron y funcionan.

## Conclusiones

ALERTA muestra que una regla de alerta temprana puede escribirse en un lenguaje formal que una máquina valide. El AFD de 17 estados reconoce todos los tokens y justifica cada estado. La ambigüedad de G0 se eliminó con una jerarquía de operadores, y la recursividad por la izquierda

y los prefijos comunes se eliminaron con transformaciones demostradas, de modo que G es LL(1) y no es ambigua. El parser sigue esa gramática regla por regla.

El trabajo tiene límites: no hay análisis semántico, y un error dentro de una condición puede producir un segundo error por el bloque que queda sin cerrar. El paso siguiente sería agregar una tabla de símbolos y un intérprete que ejecute las reglas con lecturas simuladas.

## Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Addison Wesley.

Aristizábal, E., Riaño, F., & Jiménez-Ortiz, J. (2022). Umbrales de lluvia como detonante de movimientos en masa en el piedemonte de la cordillera Central en los Andes colombianos. *Boletín de Geología*, *44*(2), 183–197. https://doi.org/10.18273/revbol.v44n2-2022009

Caine, N. (1980). The rainfall intensity-duration control of shallow landslides and debris flows. *Geografiska Annaler: Series A, Physical Geography*, *62*(1/2), 23–27. https://doi.org/10.2307/520449

Congreso de la República de Colombia. (2012, 24 de abril). *Ley 1523 de 2012: Por la cual se adopta la política nacional de gestión del riesgo de desastres y se establece el Sistema Nacional de Gestión del Riesgo de Desastres y se dictan otras disposiciones*. Diario Oficial No. 48.411. https://normas.cra.gov.co/gestor/docs/ley_1523_2012.htm

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3rd ed.). Addison-Wesley.

La FM. (2017, 3 de diciembre). *Ideam lanzó alertas a nivel nacional por riesgo de inundaciones y deslizamientos*. https://www.lafm.com.co/colombia/ideam-lanzo-alertas-nivel-nacional-por-riesgo-de-inundaciones-y-deslizamientos

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.